

ID607001: Introductory Application Development Concepts

Project Marking Rubric

Functionality - Learning Outcome 1 (50%)

	A – 5-4	B – 3.5	C – 3-2.5	D/E – 2-0
Models (5%)	All ten models implemented with at least five fields each. User model properly implemented. Excellent range of data types used. All five relationships between models implemented correctly. All three enums implemented with at least three values each. Models demonstrate sophisticated design and optimal structure.	All ten models implemented with required fields. Good range of data types. At least four relationships between models implemented. All three enums implemented with required values. Models demonstrate good design with minor issues.	Eight to nine models implemented with required fields. Adequate range of data types. Three to four relationships implemented. Two to three enums implemented. Models demonstrate adequate design with some issues.	Fewer than eight models implemented or most models lack required fields. Limited range of data types. Fewer than three relationships implemented. Fewer than two enums implemented. Models demonstrate poor design with major issues.
	A – 10-8	B – 7.5-6.5	C – 6-5	D/E – 4.5-0
Authentication and Role-Based Access Control (10%)	All three roles (SUPER_ADMIN, ADMIN, NORMAL) properly implemented. Registration functions correctly for NORMAL users only. Login functionality works flawlessly for all users. User locking after five failed attempts works perfectly with proper timeout. Logout with token blacklisting fully implemented and working correctly. Authentication flow is secure and robust with excellent error handling.	All roles implemented. Registration and login functioning correctly. User locking implemented but with minor timing or counter issues. Logout with token blacklisting implemented with minor issues. Authentication flow is secure with good error handling.	Roles implemented but with some issues. Basic registration and login functioning. User locking implemented but with significant issues. Token blacklisting implemented but not fully functional. Authentication flow has security vulnerabilities or substantial error handling issues.	Incomplete role implementation. Registration or login non-functional or insecure. User locking missing or seriously flawed. Token blacklisting missing or non-functional. Major security concerns in authentication flow.
	A – 15-12	B – 11.5-10	C – 9.5-7.5	D/E – 7-0

CRUD Operations (15%)	Repository, controller, and route files correctly implemented for all models. All CRUD operations (getAll, getByid, create, update, delete) implemented and working perfectly. Comprehensive request body validation with four rules per field. All fields filterable and sortable in both directions. Pagination works correctly with default of 25 items per page. All role-based permissions (SUPER_ADMIN, ADMIN, NORMAL) implemented perfectly according to specifications.	Repository, controller and route files implemented for most models. All CRUD operations implemented with minor issues. Good request validation with most fields having required rules. Filtering and sorting implemented with minor issues. Pagination works with correct defaults. Role-based permissions implemented with minor inconsistencies.	Repository, controller and route structure implemented but with some gaps. Basic CRUD operations working but with significant issues. Request validation implemented but incomplete. Basic filtering, sorting, or pagination implemented but with significant limitations. Role-based permissions partially implemented with notable gaps.	Significant gaps in repository, controller, or route implementation. Multiple CRUD operations missing or non-functional. Minimal or no request validation. Filtering, sorting, or pagination missing or severely limited. Role-based permissions poorly implemented or missing key components.
	A – 10-8	B – 7.5-6.5	C – 6-5	D/E – 4.5-0
Seeding and API Tests (10%)	All seeding requirements perfectly implemented: five NORMAL users via CSV, five ADMIN users via JSON, five SUPER_ADMIN users via GitHub Gist. Comprehensive API tests covering all CRUD operations, filtering, sorting, pagination, authentication, and request limiting for all models. Tests are well-organised and thoroughly cover edge cases.	All required seeding implemented with minor issues. API tests cover most requirements with good organisation but miss some edge cases or have minor implementation issues.	Most seeding requirements implemented but with significant issues. Basic API tests implemented but lacking coverage for some major functionality or with significant implementation issues.	Seeding implementation incomplete or significantly flawed. API tests missing major components or non-functional.
	A – 5-4	B – 3.5	C – 3-2.5	D/E – 2-0

Logging and Security (5%)	All logging requirements implemented perfectly to combined.log. X-Powered-By header properly secured. Request limiting for both read and write operations implemented exactly to specification with correct rates, windows, and error messages. Security implementations are robust and comprehensive.	Logging to file implemented with minor issues. X-Powered-By header secured. Request limiting implemented for both operation types but with minor deviations from specifications. Overall security implementation is good.	Basic logging implemented but with significant limitations. X-Powered-By header addressed. Request limiting implemented but with significant deviations from specifications. Security implementation has notable gaps.	Logging missing or non-functional. X-Powered-By header not secured. Request limiting absent or severely limited. Major security vulnerabilities present.
	A – 5-4	B – 3.5	C – 3-2.5	D/E – 2-0
Development, Testing and Production (5%)	Environment variables correctly stored in .env file. API base URL properly configured as /api/v1. Docker correctly configured for PostgreSQL in development and testing. Render properly configured for PostgreSQL in production. REST API successfully deployed as a web service on Render. Deployment pipeline is robust and well-documented.	Environment variables managed correctly. API base URL configured properly. Docker and Render configurations functional with minor issues. REST API deployed with minor configuration issues.	Environment variable management implemented but with limitations. API base URL configured. Basic Docker and Render configuration present but with significant issues. REST API deployment functional but with substantial configuration problems.	Environment variable management missing or severely flawed. API base URL misconfigured. Docker or Render configurations missing or non-functional. REST API deployment unsuccessful or with critical issues.

Code Quality and Best Practices - Learning Outcome 1 (40%)

	A – 10-8	B – 7.5-6.5	C – 6-5	D/E – 4.5-0
Project Structure (10%)	Exceptional project structure with clear separation of concerns. Files and directories are logically organised with consistent patterns. Models, controllers, repositories, and routes are perfectly separated. Code is highly modular with excellent reusability. Dependencies are well-managed with clear import/export patterns. Structure demonstrates expert understanding of software architecture principles.	Good project structure with clear separation of concerns. Logical organisation of files and directories with minor inconsistencies. Good modularity and reusability. Well-managed dependencies with occasional minor issues in import/export patterns. Structure demonstrates good understanding of software architecture principles.	Acceptable project structure with basic separation of concerns. Some logical organisation but with notable inconsistencies. Limited modularity and reusability. Dependencies managed but with some significant issues. Structure demonstrates basic understanding of software architecture principles.	Poor project structure with minimal separation of concerns. Disorganised files and directories with major inconsistencies. Little to no modularity or reusability. Poorly managed dependencies with critical issues. Structure demonstrates limited understanding of software architecture principles.
	A – 5-4	B – 3.5	C – 3-2.5	D/E – 2-0
Naming Conventions (5%)	Exceptional naming conventions throughout the codebase. All files, classes, functions, variables, and resource groups have clear, descriptive, and consistent names that perfectly reflect their purpose. Naming follows industry standard conventions (camelCase, PascalCase, etc.) appropriately throughout all code. Naming demonstrates exceptional clarity and consistency.	Good naming conventions with minor inconsistencies. Most files, classes, functions, variables, and resource groups have clear and descriptive names. Naming mostly follows appropriate conventions with occasional deviations. Good overall clarity and consistency in naming.	Acceptable naming conventions but with notable inconsistencies. Some files, classes, functions, variables, and resource groups lack clarity or descriptiveness. Inconsistent application of naming conventions. Basic level of clarity in naming with significant room for improvement.	Poor naming conventions throughout. Many files, classes, functions, variables, and resource groups have unclear, non-descriptive, or misleading names. Little to no consistent application of naming conventions. Names create confusion and hinder code readability.
	A – 5-4	B – 3.5	C – 3-2.5	D/E – 2-0

Documentation and Comments (5%)	Exceptional documentation using JSDoc format throughout the codebase. All classes, functions, and complex sections have comprehensive, clear comments that perfectly explain their purpose, parameters, return values, and behaviour. Comments on complex logic are insightful and helpful. Documentation consistently maintained across the entire codebase.	Good documentation using JSDoc format with minor gaps. Most classes, functions, and complex sections have good comments. Comments generally explain purpose and behaviour well. Documentation maintained across most of the codebase with occasional inconsistencies.	Acceptable documentation but with significant gaps. Basic JSDoc format used inconsistently. Some classes, functions, and complex sections lack comments. Existing comments provide basic explanation but may miss important details. Documentation quality varies considerably across the codebase.	Poor or minimal documentation. Little to no use of JSDoc format. Many classes, functions, and complex sections lack comments. Existing comments are sparse, unhelpful, or outdated. Documentation is inconsistent or missing from large portions of the codebase.
	A – 5-4	B – 3.5	C – 3-2.5	D/E – 2-0
Code Formatting (5%)	Exceptional code formatting throughout the codebase. Perfect consistency in indentation, spacing, and use of braces. Appropriate blank lines between sections for optimal readability. Proper use of Prettier or similar tool with evidence of automated formatting in all files. Configuration files for formatting tools are properly set up and maintained.	Good code formatting with minor inconsistencies. Generally consistent indentation, spacing, and use of braces. Appropriate use of blank lines in most places. Evidence of automated formatting tool usage with occasional manual overrides or missed files.	Acceptable code formatting but with notable inconsistencies. Basic formatting standards followed but with significant variations. Inconsistent use of blank lines. Some evidence of automated formatting tools but applied inconsistently.	Poor code formatting throughout. Inconsistent or improper indentation, spacing, and use of braces. Minimal use of blank lines or inappropriate placement. Little to no evidence of automated formatting tools. Code is difficult to read due to formatting issues.
	A – 5-4	B – 3.5	C – 3-2.5	D/E – 2-0

Dead Code (5%)	No dead code present in the codebase. All files, classes, functions, variables, and resource groups serve a clear purpose. No commented-out code blocks, unused imports, or redundant declarations. Code demonstrates exceptional discipline in maintaining only necessary components. Evidence of regular code reviews or linting to eliminate dead code.	Minimal dead code with minor instances. Few unused elements that don't significantly impact codebase clarity. Rare instances of commented-out code with clear explanatory notes. Good discipline in maintaining code cleanliness with occasional oversights.	Some dead code present throughout the codebase. Multiple instances of unused elements that affect codebase clarity. Several commented-out code blocks without clear explanation. Basic attention to code cleanliness with notable room for improvement.	Significant dead code throughout the codebase. Many unused or redundant files, classes, functions, variables, or resource groups. Numerous commented-out code blocks without explanation. Little attention paid to code cleanliness, creating confusion and potential maintenance issues.
	A – 10-8	B – 7.5-6.5	C – 6-5	D/E – 4.5-0
Performance and Scalability (10%)	Exceptional attention to performance and scalability. Optimal algorithm selection and implementation. Efficient data structures used consistently. Code demonstrates expert understanding of performance optimisation techniques and scalability principles.	Good attention to performance and scalability. Efficient algorithms and data structures used in most cases. Code demonstrates good understanding of performance optimisation and scalability principles.	Basic attention to performance and scalability. Algorithms and data structures are functional but with room for optimisation. Code demonstrates basic understanding of performance considerations.	Poor attention to performance and scalability. Inefficient algorithms and inappropriate data structures. Code demonstrates limited understanding of performance optimisation techniques.

Documentation and Git Usage - Learning Outcome 1 (10%)

	A – 5-4	B – 3.5	C – 3-2.5	D/E – 2-0
GitHub Issues and Repository Documentation (5%)	Exceptional and consistent use of GitHub issues throughout the entire development process. Issues are well-organised, properly labeled, and contain detailed descriptions. Progress is clearly tracked with regular updates and appropriate issue closure. Comprehensive README with all required elements perfectly documented: REST API URL, development and testing environment setup instructions, database seeding instructions, API testing instructions, Prisma Studio access, clear ERD diagram, and video demonstration URL. Documentation is exceptionally well-organised and easy to follow. Expert use of Markdown formatting with appropriate headings, lists, tables, code blocks, links, and emphasis. Documentation is visually appealing and highly readable. Flawless spelling and grammar throughout all documentation.	Good and generally consistent use of GitHub issues throughout development. Issues are organised with appropriate labels and descriptions. Progress is tracked with regular updates. Good README containing most required elements with clear documentation. Minor omissions or areas that could be improved in clarity. Setup instructions are functional but may have minor gaps. Good use of Markdown formatting with appropriate structure. Some advanced formatting features utilised. Good spelling and grammar with only minor errors that do not impact understanding.	Basic use of GitHub issues but with inconsistent application. Issues have minimal descriptions and limited organisation. Progress tracking is present but sporadic. Basic README containing some required elements but with significant omissions or unclear instructions. Setup procedures may be difficult to follow. Basic use of Markdown with simple formatting. Limited use of advanced features. Multiple spelling and grammar errors that occasionally impact clarity but do not prevent understanding.	Minimal or inadequate use of GitHub issues. Many issues lack descriptions or organisation. Little to no evidence of progress tracking. Poor or incomplete README missing multiple required elements. Instructions are unclear, incorrect, or missing entirely. Poor use of Markdown with little to no formatting. Documentation is difficult to read due to formatting issues. Numerous spelling and grammar errors that significantly impact clarity and understanding.
	A – 5-4	B – 3.5	C – 3-2.5	D/E – 2-0

Git Configuration and Commit Practices (5%)	Perfect implementation of the specified .gitignore file from GitHub. No unnecessary files are tracked, and all appropriate files are ignored. Repository is free from any IDE configurations, node_modules, build artifacts, or environment files. Exceptional commit messages that perfectly reflect the context of each functional requirement change. Messages follow a consistent and appropriate naming convention style (e.g., conventional commits). Commit history tells a clear story of project development with logical progression. Commits are appropriately sized with related changes grouped together.	Good implementation of the specified .gitignore file with minor oversights. Few unnecessarily tracked files. Repository is generally clean from system or build files. Good commit messages that generally reflect functional changes. Messages follow a naming convention with occasional inconsistencies. Commit history shows good progression of development. Commit sizes are generally appropriate.	Basic implementation of the specified .gitignore file with some significant oversights. Some unnecessary files are tracked. Repository contains some files that should be ignored. Basic commit messages that sometimes reflect functional changes. Inconsistent application of naming conventions. Commit history provides basic understanding of development progression. Commit sizes vary widely with some too large or too small.	Poor implementation of the specified .gitignore file or no custom .gitignore used. Many unnecessary files are tracked. Repository is cluttered with system or build files. Poor commit messages that rarely reflect functional changes. No consistent naming convention. Commit history does not clearly show development progression. Commits are inappropriately sized (too large or too frequent with minimal changes).
---	--	---	--	--

ID607001: Introductory Application Development Concepts

Project Marking Cover Sheet

Name:

Date:

Learner ID:

Assessor's Name:

Assessor's Signature:

Criteria	Weighting	Mark
Functionality	50	
Code Quality and Best Practices	40	
Documentation and Git Usage	10	
Final Result	/100	
This assessment is worth 80% of the final mark for the Introductory Application Development Concepts course.		

Feedback:

Functionality:

Code Quality and Best Practices:

Documentation and Git Usage: